

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM**

**MODUL 6 & 7 MOBILE (PERTEMUAN 11)
INTERAKSI PENGGUNA & NAVIGASI DAN NOTIFIKASI**



Disusun oleh:

MUHAMMAD DHIMAS HAFIZH FATHURRAHMAN

2311102151

PSIF-11-REG04

Dosen Pengampu:

CAHYO PRIHANTORO, S.Kom., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

LINK GIT :

https://github.com/Masdim37/2311102151_M-Dhimas-Hafizh-F_Repository-ABP/tree/master/Pertemuan-11

Modul 6 & 7 Mobile (Pertemuan 11)

Interaksi Pengguna & Navigasi dan Notifikasi

Tugas Pertemuan 11 :

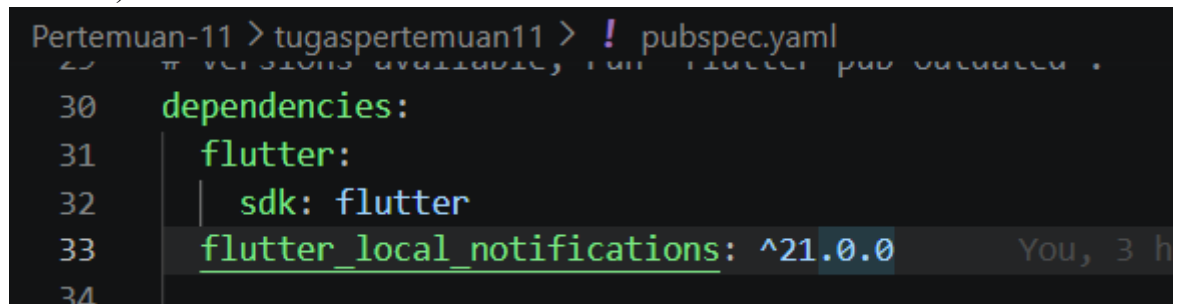
Membaca modul 7 dan buat route dari project flutter yang terdiri dari beberapa halaman dan tambahkan notifikasi. Untuk notifikasinya bebas modelnya mau kaya gimana.

JAWABAN

a. Langkah-langkah pengerjaan :

1. Menambahkan dependencies

Menambahkan flutter_local_notifications pada pubspec.yaml agar aplikasi Flutter memiliki kemampuan untuk mengakses dan menampilkan notifikasi lokal secara langsung pada antarmuka sistem perangkat (seperti status bar atau layar atas pada Android).



```
Pertemuan-11 > tugaspertemuan11 > ! pubspec.yaml
29  # versions available, run flutter pub update .
30  dependencies:
31    flutter:
32      sdk: flutter
33    flutter_local_notifications: ^21.0.0
34
```

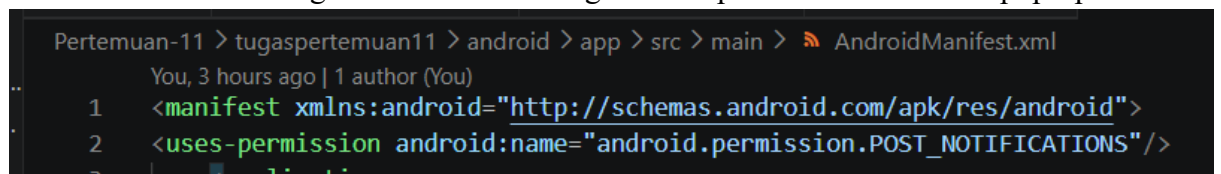
2. Konfigurasi Native Android & Mengatasi Konflik Gradle

menyesuaikan sistem build Android agar mendukung fitur Java modern (Java 8+) yang diadopsi oleh package notifikasi versi terbaru.

Tambahkan izin <uses-permission

android:name="android.permission.POST_NOTIFICATIONS"/> pada

AndroidManifest.xml agar Android 13+ mengizinkan aplikasi memunculkan pop-up.



```
Pertemuan-11 > tugaspertemuan11 > android > app > src > main > AndroidManifest.xml
You, 3 hours ago | 1 author (You)
1  <manifest xmlns:android="http://schemas.android.com/apk/res/android">
2  <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
3  <application
```

Pada android/app/build.gradle.kts aktifkan fitur desugaring

(isCoreLibraryDesugaringEnabled = true) dan pasang versi pustaka desugar terbaru

(coreLibraryDesugaring("com.android.tools.desugar_jdk_libs:2.1.4"))

```
plugins {
    id("com.android.application")
    id("kotlin-android")
    // The Flutter Gradle Plugin must be applied after the Android and Kotlin
    Gradle plugins.
```

```

    id("dev.flutter.flutter-gradle-plugin")
}

android {
    namespace = "com.example.tugaspertemuan11"
    compileSdk = flutter.compileSdkVersion
    ndkVersion = flutter.ndkVersion

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
        isCoreLibraryDesugaringEnabled = true
    }

    kotlinOptions {
        jvmTarget = JavaVersion.VERSION_17.toString()
    }

    defaultConfig {
        // TODO: Specify your own unique Application ID
        (https://developer.android.com/studio/build/application-id.html).
        applicationId = "com.example.tugaspertemuan11"
        // You can update the following values to match your application needs.
        // For more information, see: https://flutter.dev/to/review-gradle-config.
        minSdk = flutter.minSdkVersion
        targetSdk = flutter.targetSdkVersion
        versionCode = flutter.versionCode
        versionName = flutter.versionName
    }

    buildTypes {
        release {
            // TODO: Add your own signing config for the release build.
            // Signing with the debug keys for now, so `flutter run --release` works.
            signingConfig = signingConfigs.getByName("debug")
        }
    }
}

flutter {
    source = "../.."
}

dependencies {
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.1.4")
}

```

3. Buat model class

Model class yang akan dibuat adalah model mahasiswa (lib/model_mahasiswa.dart).

```
Pertemuan-11 > tugaspertemuan11 > lib > model_mahasiswa.dart > ...
You, 48 seconds ago | 1 author (You)
1  class Mahasiswa {
2      String nama;
3      String nim;
4      String jurusan;
5      double ipk;
6      DateTime tgllahir;
7      bool gender;
8
9      Mahasiswa({
10         required this.nama,
11         required this.nim,
12         required this.jurusan,
13         required this.ipk,
14         required this.tgllahir,
15         required this.gender,
16     });
17 }
18 You, 3 seconds ago • Uncommitted changes
```

Penjelasan :

Kode diatas adalah implementasi model class bernama Mahasiswa yang dirancang menggunakan pendekatan Object-Oriented Programming (OOP) untuk bertindak sebagai wadah data (data container) tunggal yang mengikat berbagai variabel visual dari komponen interaksi pengguna seperti TextFormField untuk nama, nim, dan ipk, DropdownButtonFormField untuk jurusan, showDatePicker untuk tanggal lahir, serta Radio button untuk gender menjadi satu kesatuan objek yang aman (type-safe), yang kemudian digunakan sebagai objek dinamis yang disimpan ke dalam memori, dilempar antar halaman melalui argumen rute Navigator, serta diekstraksi properti teksnya untuk ditampilkan secara personal pada bar local notifications sistem Android.

4. Membuat menu utama & pengikat state (main.dart)

Buat menu utama pada file main.dart. file ini dirancang sebagai pusat kendali navigasi (Single Source of Truth) untuk mengelola alur data dan perpindahan halaman.

```
import 'package:flutter/material.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'crud_page.dart';
import 'detail_page.dart';
import 'model_mahasiswa.dart';
import 'statistik_page.dart';

//Deklarasi Variabel Global yang dapat diakses oleh file lain
final FlutterLocalNotificationsPlugin flutterLocalNotificationsPlugin =
    FlutterLocalNotificationsPlugin();
final List<String> logNotifikasi = []; //List penyimpan log riwayat aktivitas
```

```

final List<Mahasiswa> dataMahasiswa =
  []; //List dataMahasiswa untuk menyimpan data mahasiswa

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  const AndroidInitializationSettings initializationSettingsAndroid =
    AndroidInitializationSettings('@mipmap/ic_launcher');
  const InitializationSettings initializationSettings = InitializationSettings(
    android: initializationSettingsAndroid,
  );
  await flutterLocalNotificationsPlugin.initialize(
    settings: initializationSettings,
  );

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'CRUD & Notification Log',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
      ),
      initialRoute: '/',
      routes: {
        //routing halaman
        '/': (context) => const MainNavigation(),
        '/detail': (context) => const DetailPage(),
      },
    );
  }
}

class MainNavigation extends StatefulWidget {
  const MainNavigation({super.key});

  @override
  State<MainNavigation> createState() => _MainNavigationState();
}

class _MainNavigationState extends State<MainNavigation> {
  int _selectedIndex = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(

```

```

body: IndexedStack(
  index: _selectedIndex,
  children: [
    CrudMahasiswaPage(
      data: dataMahasiswa,
    ), //kirim dataMahasiswa ke halaman crud
    RiwayatLogPage(),
    StatistikPage(
      data: dataMahasiswa,
    ), //kirim dataMahasiswa ke halaman statistik
  ],
),
bottomNavigationBar: BottomNavigationBar(
  currentIndex: _selectedIndex,
  onTap: (index) {
    setState(() {
      _selectedIndex = index;
    });
  },
  items: const [
    BottomNavigationBarItem(icon: Icon(Icons.people), label: 'Mahasiswa'),
    BottomNavigationBarItem(
      icon: Icon(Icons.history),
      label: 'Riwayat Log',
    ),
    BottomNavigationBarItem(
      icon: Icon(Icons.analytics),
      label: 'Statistik',
    ),
  ],
),
);
}
}

```

//WIDGET HALAMAN MENU KEDUA: LOG RIWAYAT NOTIFIKASI

```

class RiwayatLogPage extends StatelessWidget {
  const RiwayatLogPage({super.key});

```

```

  @override

```

```

  Widget build(BuildContext context) {

```

```

    return Scaffold(

```

```

      appBar: AppBar(

```

```

        title: const Text(

```

```

          'Riwayat Aktivitas',

```

```

          style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),

```

```

        ),

```

```

        backgroundColor: Colors.amber,

```

```

      ),

```

```

      body: logNotifikasi.isEmpty

```

```

? const Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Icon(Icons.history_toggle_off, size: 60, color: Colors.grey),
      SizedBox(height: 10),
      Text(
        'Belum ada riwayat aktivitas sistem.',
        style: TextStyle(color: Colors.grey),
      ),
    ],
  ),
)
: ListView.builder(
  itemCount: logNotifikasi.length,
  itemBuilder: (context, index) {
    return Card(
      margin: const EdgeInsets.symmetric(
        horizontal: 12,
        vertical: 4,
      ),
      child: ListTile(
        leading: const Icon(
          Icons.circle_notifications,
          color: Colors.amber,
        ),
        title: Text(
          logNotifikasi[index],
          style: const TextStyle(
            fontFamily: 'monospace',
            fontSize: 13,
          ),
        ),
      ),
    );
  },
);
);
}
}

```

Penjelasan :

Kode diatas adalah pusat kendali utama (entry point) dari aplikasi Flutter yang mengintegrasikan seluruh fungsionalitas seperti mengatur inisialisasi sistem, manajemen state global, pendistribusian data, serta alur navigasi (routing) antar halaman secara terpusat.

Berikut adalah rincian peran dari masing-masing bagian pada kode main.dart :

- Deklarasi Variabel Global: Pada bagian atas, terdapat flutterLocalNotificationsPlugin, logNotifikasi, dan dataMahasiswa. Ini bertindak sebagai Single Source of Truth (sumber data tunggal). Dengan menaruh list mahasiswa di sini, halaman CRUD dan halaman Statistik akan selalu membaca dan mengolah data yang sama secara sinkron.
- Inisialisasi Sistem (main): Fungsi asinkron ini memanggil WidgetsFlutterBinding.ensureInitialized() untuk memastikan sistem Flutter siap, lalu melakukan konfigurasi awal (pengaturan ikon launcher) agar plugin notifikasi lokal Android bisa berjalan sempurna sebelum UI aplikasi dirender.
- Pengaturan Routing (MyApp): Widget ini adalah akar aplikasi yang menggunakan MaterialApp. Pada bagian ini diterapkan teknik Named Routes. Rute / diarahkan ke menu Bottom Navigation, sedangkan rute /detail didaftarkan untuk memfasilitasi Navigator.pushNamed dari halaman CRUD ke halaman Detail Mahasiswa.
- Navigasi Persisten (MainNavigation & IndexedStack): Ini adalah implementasi tingkat lanjut untuk menjaga data. Daripada menggunakan pergantian halaman biasa yang akan mereset data, kode ini menggunakan IndexedStack. Fungsinya adalah untuk menumpuk ketiga halaman (CRUD, Riwayat, Statistik) di dalam memori, sehingga ketika berpindah tab di BottomNavigationBar, data mahasiswa yang sudah diinput tidak akan hilang atau terhapus.
- Distribusi Data (Passing Data): Pada bagian children di dalam IndexedStack, variabel global dataMahasiswa dioper (passing) ke dalam CrudMahasiswaPage(data: dataMahasiswa) dan StatistikPage(data: dataMahasiswa). Inilah yang membuat halaman statistik bisa otomatis menghitung IPK dan total mahasiswa berdasarkan inputan di halaman CRUD.
- Halaman Log Riwayat (RiwayatLogPage): Widget ini secara dinamis membaca isi dari list logNotifikasi. Menggunakan Ternary Operator (? :), halaman ini menampilkan pesan "Belum ada riwayat" jika list kosong, namun akan merender daftar kartu notifikasi menggunakan ListView.builder jika pengguna sudah melakukan aksi tambah/edit/hapus data.

5. Menambahkan Fitur Manajemen Data (CRUD)

Buat fitur CRUD pada file crud_page.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'model_mahasiswa.dart';
import 'main.dart';

class CrudMahasiswaPage extends StatefulWidget {
  final List<Mahasiswa> data;
  const CrudMahasiswaPage({super.key, required this.data});

  @override
  State<CrudMahasiswaPage> createState() => _CrudMahasiswaPageState();
}
```

```

class _CrudMahasiswaPageState extends State<CrudMahasiswaPage> {
  final _formKey = GlobalKey<FormState>();
  final _namaController = TextEditingController();
  final _nimController = TextEditingController();
  final _ipkController = TextEditingController();

  String? _jurusanTerpilih;
  final List<String> _daftarJurusan = [
    'Teknik Informatika',
    'Sistem Informasi',
    'Software Engineering',
    'Sains Data',
    'Teknik Telekomunikasi',
    'Teknik Elektro',
    'Teknik Biomedis',
    'Teknik Industri',
    'Teknik Logistik',
    'Desain Komunikasi Visual',
    'Desain Produk',
    'Teknologi Pangan',
  ];

  DateTime? _tglLahirTerpilih;
  bool _genderTerpilih = true;

  //Memunculkan Notifikasi & Mencatat ke Riwayat Log
  Future<void> _tampilkanNotifikasi(String judul, String pesan) async {
    const AndroidNotificationDetails androidDetails =
      AndroidNotificationDetails(
        'crud_channel',
        'Notifikasi CRUD',
        importance: Importance.max,
        priority: Priority.high,
      );
    const NotificationDetails platformDetails = NotificationDetails(
      android: androidDetails,
    );

    //Tembakkan notifikasi ke sistem bar Android
    await flutterLocalNotificationsPlugin.show(
      id: DateTime.now().millisecondsSinceEpoch ~/ 1000,
      title: judul,
      body: pesan,
      notificationDetails: platformDetails,
    );

    //Tambahkan ke Log Riwayat
    String waktu =

```

```

        "${DateTime.now().hour.toString().padLeft(2,
'0')}.${DateTime.now().minute.toString().padLeft(2, '0')}}";
        logNotifikasi.insert(
            0,
            "[$waktu] $judul: $pesan",
        ); //menyisipkan log baru di urutan paling atas
    }

//Dialog Form (Tambah & Edit Data)
void _tampilFormDialog({int? index}) {
    if (index != null) {
        //Mode Edit: Isi form dengan data lama yang dipilih
        _namaController.text = widget.data[index].nama;
        _nimController.text = widget.data[index].nim;
        _ipkController.text = widget.data[index].ipk.toString();
        _jurusanTerpilih = widget.data[index].jurusan;
        _tglLahirTerpilih = widget.data[index].tglLahir;
        _genderTerpilih = widget.data[index].gender;

        _namaController.selection = TextSelection.fromPosition(
            TextPosition(offset: _namaController.text.length),
        );
        _nimController.selection = TextSelection.fromPosition(
            TextPosition(offset: _nimController.text.length),
        );
        _ipkController.selection = TextSelection.fromPosition(
            TextPosition(offset: _ipkController.text.length),
        );
    } else {
        //MODE CREATE: Kosongkan Form
        _namaController.clear();
        _nimController.clear();
        _ipkController.clear();
        _jurusanTerpilih = null;
        _tglLahirTerpilih = null;
        _genderTerpilih = true;
    }

    showDialog(
        context: context,
        builder: (context) => StatefulBuilder(
            builder: (context, setDialogState) => AlertDialog(
                title: Text(
                    index == null ? 'Tambah Mahasiswa' : 'Edit Mahasiswa',
                    style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),
                ),
                content: SingleChildScrollView(
                    child: Form(
                        key: _formKey,
                        child: Column(

```

```

mainAxisSize: MainAxisSize.min,
children: [
  TextFormField(
    controller: _namaController,
    decoration: const InputDecoration(
      labelText: 'Nama Lengkap',
    ),
    validator: (val) => val!.isEmpty ? 'Wajib diisi' : null,
  ),
  TextFormField(
    controller: _nimController,
    decoration: const InputDecoration(labelText: 'NIM'),
    keyboardType: TextInputType.number,
    validator: (val) => val!.isEmpty ? 'Wajib diisi' : null,
  ),
  DropdownButtonFormField<String>(
    value: _jurusanTerpilih,
    decoration: const InputDecoration(
      labelText: 'Program Studi',
    ),
    items: _daftarJurusan
      .map((j) => DropdownMenuItem(value: j, child: Text(j)))
      .toList(),
    onChanged: (val) =>
      setState(() => _jurusanTerpilih = val),
    validator: (val) => val == null ? 'Pilih jurusan' : null,
  ),
  TextFormField(
    controller: _ipkController,
    decoration: const InputDecoration(labelText: 'IPK'),
    keyboardType: const TextInputType.numberWithOptions(
      decimal: true,
    ),
    validator: (val) => val!.isEmpty ? 'Wajib diisi' : null,
  ),
  const SizedBox(height: 15),
  Row(
    children: [
      Expanded(
        child: Text(
          _tglLahirTerpilih == null
            ? 'Tanggal Lahir'
            : 'Tanggal Lahir:
$ {_tglLahirTerpilih!.day} / $ {_tglLahirTerpilih!.month} / $ {_tglLahirTerpilih!.year}
}',
        ),
      ),
      TextButton(
        onPressed: () async {
          DateTime? picked = await showDatePicker(

```

```

        context: context,
        initialDate: _tglLahirTerpilih ?? DateTime.now(),
        firstDate: DateTime(1990),
        lastDate: DateTime.now(),
    );
    if (picked != null)
        setDialogState(() => _tglLahirTerpilih = picked);
  },
  child: const Text('Pilih'),
),
],
),
const Align(
  alignment: Alignment.centerLeft,
  child: Text(
    'Jenis Kelamin',
    style: TextStyle(fontWeight: FontWeight.bold),
  ),
),
Row(
  children: [
    Radio<bool>(
      value: true,
      groupValue: _genderTerpilih,
      onChanged: (val) =>
        setDialogState(() => _genderTerpilih = val!),
    ),
    const Text('Laki-laki'),
    Radio<bool>(
      value: false,
      groupValue: _genderTerpilih,
      onChanged: (val) =>
        setDialogState(() => _genderTerpilih = val!),
    ),
    const Text('Perempuan'),
  ],
),
],
),
),
actions: [
  TextButton(
    onPressed: () => Navigator.pop(context),
    child: const Text('Batal'),
  ),
  ElevatedButton(
    onPressed: () {
      if (_formKey.currentState!.validate() &&
        _tglLahirTerpilih != null) {

```

```

        setState(() {
          final mhsBaru = Mahasiswa(
            nama: _namaController.text,
            nim: _nimController.text,
            jurusan: _jurusanTerpilih!,
            ipk: double.parse(_ipkController.text),
            tgllahir: _tglLahirTerpilih!,
            gender: _genderTerpilih,
          );

          if (index == null) {
            //notifikasi ketika Create Data
            widget.data.add(mhsBaru);
            _tampilkanNotifikasi(
              "Data Berhasil Ditambah!",
              "Mahasiswa ${_namaController.text} ditambahkan.",
            );
          } else {
            //notifikasi ketika Edit/Update Data
            widget.data[index] = mhsBaru;
            _tampilkanNotifikasi(
              "Data Berhasil Diperbarui!",
              "Data ${_namaController.text} telah diubah.",
            );
          }
        });
        Navigator.pop(context);
      },
      child: const Text('Simpan'),
    ),
  ],
),
),
);
}

//Dialog Form (Delete Data)
void _hapusData(int index) {
  String namaDihapus = widget.data[index].nama;
  setState(() {
    widget.data.removeAt(index);
  });
  //notifikasi ketika delete Data
  _tampilkanNotifikasi(
    "Data Dihapus",
    "Data mahasiswa bernama $namaDihapus dihapus dari sistem.",
  );
}

```

```

//Function Dialog Konfirmasi Sebelum Hapus
void _konfirmasiHapus(int index) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Row(
        children: [
          Icon(Icons.warning, color: Colors.orange),
          SizedBox(width: 10),
          Text('Konfirmasi Hapus'),
        ],
      ),
      content: Text(
        'Apakah Anda yakin ingin menghapus data dari mahasiswa bernama
        ${widget.data[index].nama}?',
      ),
      actions: [
        //pilihan 1: Batal (Tutup dialog tanpa menghapus)
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('Batal'),
        ),
        //pilihan 2: Ya (Tutup dialog lalu jalankan proses hapus)
        ElevatedButton(
          style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
          onPressed: () {
            Navigator.pop(context); //tutup dialog konfirmasi terlebih dahulu
            _hapusData(index); //jalankan fungsi hapus data & notifikasi
          },
          child: const Text(
            'Ya, Hapus',
            style: TextStyle(color: Colors.white),
          ),
        ),
      ],
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text(
        'Data Mahasiswa',
        style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),
      ),
      backgroundColor: Colors.blue,
    ),
    body: widget.data.isEmpty

```

```

? const Center(child: Text('Belum ada data mahasiswa.'))
: ListView.builder(
  itemCount: widget.data.length,
  itemBuilder: (context, index) {
    final mhs = widget.data[index];
    return Card(
      margin: const EdgeInsets.symmetric(
        horizontal: 10,
        vertical: 5,
      ),
      child: ListTile(
        leading: Icon(Icons.people),
        title: Text(
          mhs.nama,
          style: const TextStyle(fontWeight: FontWeight.bold),
        ),
        subtitle: Text('NIM : ${mhs.nim}'),
        onTap: () {
          Navigator.pushNamed(context, '/detail', arguments: mhs);
        },
        //menampilkan tombol edit & delete
        trailing: Row(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            IconButton(
              icon: const Icon(Icons.edit, color: Colors.blue),
              onPressed: () => _tampilFormDialog(index: index),
            ),
            IconButton(
              icon: const Icon(Icons.delete, color: Colors.red),
              onPressed: () => _konfirmasiHapus(index),
            ),
          ],
        ),
      ),
    );
  },
),
floatingActionButton: FloatingActionButton(
  onPressed: () => _tampilFormDialog(),
  child: const Icon(Icons.add),
),
);
}
}

```

Penjelasan :

Kode diatas adalah implementasi inti dari halaman manajemen data yang mengkolaborasikan secara penuh fungsionalitas antarmuka interaktif dan sistem kendali, di mana halaman ini bertugas sebagai area utama bagi pengguna untuk

melakukan operasi CRUD (Create, Read, Update, Delete) melalui serangkaian komponen input kompleks yang datanya dikonversi menjadi objek model terstruktur, sekaligus dilengkapi dengan mekanisme perpindahan rute navigasi dan pemicu notifikasi lokal sistem.

Berikut adalah rincian peran dari masing-masing bagian pada kode `crud_page.dart` :

- **Penerimaan Data Global (State Terpusat):** Kelas `CrudMahasiswaPage` menerima parameter `List<Mahasiswa>` data yang dikirim dari `main.dart`. Hal ini memastikan bahwa halaman CRUD memanipulasi referensi data yang sama persis dengan halaman statistik, menjaga konsistensi aplikasi.
- **Interaksi Form dan Validasi:** Fungsi `_tampilFormDialog` menggunakan pembungkus Form dengan `GlobalKey` untuk memvalidasi input dari berbagai komponen: `TextFormField` (untuk teks dan angka desimal), `DropDownButtonFormField` (untuk memilih daftar Program Studi), `showDatePicker` (kalender bawaan untuk tanggal lahir), dan `Radio` (untuk opsi boolean Jenis Kelamin).
- **Manajemen Kursor (User Experience):** Pada blok logika Edit Data, kode menggunakan `TextSelection.fromPosition(TextPosition(offset: length))` agar saat dialog edit terbuka, kursor pengetikan langsung berada di akhir kata dan tidak menyeleksi (memblok) seluruh teks lama, sehingga pengguna lebih nyaman saat mengetik.
- **Pengiriman Data via Navigasi:** Pada widget `ListView.builder` yang menampilkan daftar mahasiswa, aksi klik (`onTap`) pada `ListTile` dieksekusi menggunakan `Navigator.pushNamed(context, '/detail', arguments: mhs)`. Objek Mahasiswa dilempar seutuhnya sebagai argumen ke halaman `detail_page.dart`.
- **Integrasi Notifikasi & Log Riwayat:** Fungsi `_tampilkanNotifikasi()` dipanggil setiap kali pengguna berhasil menyimpan data baru, mengedit data lama, atau menghapus data. Fungsi ini mengemas judul dan pesan ke dalam `AndroidNotificationDetails`, memunculkannya sebagai notifikasi Android (status bar), dan secara bersamaan mencatat waktu serta aksi tersebut ke dalam list `logNotifikasi` menggunakan metode `.insert(0, ...)` agar tampil di urutan teratas pada halaman riwayat.
- **Lapisan Keamanan Destruktif (AlertDialog):** Sebelum mengeksekusi penghapusan data, kode memanggil fungsi `_konfirmasiHapus()`. Fungsi ini menampilkan pop-up peringatan interaktif untuk memastikan bahwa pengguna benar-benar berniat menghapus data (opsi "Ya, Hapus" atau "Batal"), mencegah kehilangan data akibat ketidaksengajaan klik.

6. Membuat halaman detail data mahasiswa (`detail_page.dart`)

Halaman ini ditampilkan ketika menekan salah satu card mahasiswa pada halaman `main.dart`

```
import 'package:flutter/material.dart';
import 'model_mahasiswa.dart';

class DetailPage extends StatelessWidget {
```

```

const DetailPage({super.key});

@override
Widget build(BuildContext context) {
  //menangkap data (object) yang dikirim melalui argumen rute
  final Mahasiswa mhs =
    ModalRoute.of(context)!.settings.arguments as Mahasiswa;

  //mengubah boolean gender menjadi teks
  String genderTeks = mhs.gender ? "Laki-laki" : "Perempuan";
  //format tanggal lahir
  String tglLahirTeks =
    "${mhs.tglLahir.day}-${mhs.tglLahir.month}-${mhs.tglLahir.year}";

  return Scaffold(
    appBar: AppBar(
      title: const Text(
        'Detail Mahasiswa',
        style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),
      ),
      backgroundColor: Colors.blue,
    ),
    body: Padding(
      padding: const EdgeInsets.all(20.0),
      child: Card(
        elevation: 4,
        child: Padding(
          padding: const EdgeInsets.all(20.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const SizedBox(height: 10),
              Text(
                'Nama Mahasiswa : ${mhs.nama}',
                style: const TextStyle(fontSize: 18),
              ),
              const SizedBox(height: 10),
              Text('NIM : ${mhs.nim}', style: const TextStyle(fontSize: 18)),
              const SizedBox(height: 10),
              Text(
                'Program Studi : ${mhs.jurusan}',
                style: const TextStyle(fontSize: 18),
              ),
              const SizedBox(height: 10),
              Text('IPK : ${mhs.ipk}', style: const TextStyle(fontSize: 18)),
              const SizedBox(height: 10),
              Text(
                'Tanggal Lahir : $tglLahirTeks',
                style: const TextStyle(fontSize: 18),
              ),
            ],
          ),
        ),
      ),
    ),
  );
}

```

```

    ),
    const SizedBox(height: 10),
    Text(
      'Jenis Kelamin : $genderTeks',
      style: const TextStyle(fontSize: 18),
    ),
  ],
),
),
),
),
);
}
}

```

Penjelasan :

Kode diatas adalah implementasi halaman antarmuka statis (StatelessWidget) bernama `DetailPage` yang berfungsi sebagai titik akhir penerima rute navigasi, di mana halaman ini bertugas secara khusus untuk menangkap, mengekstrak, dan menerjemahkan objek model data Mahasiswa yang dikirimkan melalui argumen rute (route arguments) dari halaman utama, lalu menyajikannya menjadi sebuah tampilan kartu informasi profil visual yang rapi, lengkap, dan informatif tanpa melibatkan elemen interaksi form input.

Berikut adalah rincian peran dari masing-masing bagian pada kode `detail_page.dart` :

- **Penerimaan Argumen Navigasi** : Baris kode `ModalRoute.of(context)!.settings.arguments as Mahasiswa` merupakan inti dari implementasi pengiriman data antar halaman. Fungsi ini bekerja seperti penangkap (catcher) yang mengambil objek data utuh yang dilempar oleh `Navigator.pushNamed` pada halaman `crud_page.dart`, lalu memastikannya (casting) kembali ke dalam bentuk tipe data cetak biru aslinya, yaitu kelas `Mahasiswa`.
- **Pemformatan dan Konversi Data (Data Processing)**: Karena tidak semua tipe data di dalam model cocok untuk langsung ditampilkan kepada pengguna, halaman ini melakukan penerjemahan data mentah:
 - **Gender**: Menggunakan Ternary Operator (`mhs.gender ? "Laki-laki" : "Perempuan"`), tipe data boolean (`true/false`) yang awalnya dipilih via Radio button diubah kembali menjadi teks yang mudah dipahami manusia.
 - **Tanggal Lahir**: Objek `DateTime` dipecah menggunakan properti `.day`, `.month`, dan `.year`, lalu digabungkan (string interpolation) menjadi format penulisan tanggal standar yang rapi (`DD-MM-YYYY`).
- **Ekstraksi Properti Model Class**: Variabel penampung `mhs` digunakan untuk memanggil satu per satu atribut yang tersimpan di dalam objek tersebut (seperti `mhs.nama`, `mhs.nim`, `mhs.jurusan`, dan `mhs.ipk`). Ini membuktikan betapa praktis dan amannya menggunakan Model Class dibandingkan menggunakan Map atau Array biasa, karena terhindar dari salah ketik (typo) pemanggilan data.

- Penyajian Visual Berbasis Statis (StatelessWidget): Halaman ini dirancang menggunakan StatelessWidget karena sifat halamannya yang pasif; fungsinya murni hanya untuk merender dan menampilkan teks di atas komponen Card dan Column. Tidak ada state atau nilai yang berubah-ubah seiring berjalannya waktu setelah halaman ini dimuat ke layar.

7. Membuat halaman menu statistik (statistik_page.dart)

Halaman statistik adalah menu ketiga setelah menu utama (mahasiswa) dan menu riwayat log (riwayat notifikasi).

```
import 'package:flutter/material.dart';
import 'model_mahasiswa.dart';

class StatistikPage extends StatefulWidget {
  final List<Mahasiswa> data;
  const StatistikPage({super.key, required this.data});

  @override
  State<StatistikPage> createState() => _StatistikPageState();
}

class _StatistikPageState extends State<StatistikPage> {
  String? _selectedJurusan;

  final List<String> _daftarJurusan = [
    'Teknik Informatika',
    'Sistem Informasi',
    'Software Engineering',
    'Sains Data',
    'Teknik Telekomunikasi',
    'Teknik Elektro',
    'Teknik Biomedis',
    'Teknik Industri',
    'Teknik Logistik',
    'Desain Komunikasi Visual',
    'Desain Produk',
    'Teknologi Pangan',
  ];

  @override
  void initState() {
    super.initState();
    _selectedJurusan = _daftarJurusan[0];
  }

  @override
  Widget build(BuildContext context) {
    //Mengambil data global (Data mahasiswa dari semua jurusan)
    int totalGlobal = widget.data.length; //data total mahasiswa
    double rataIpkGlobal =
```

```

totalGlobal ==
    0 //data rata-rata IPK
? 0
: widget.data.map((m) => m.ipk).reduce((a, b) => a + b) / totalGlobal;

//mengambil data sesuai filter jurusan yang dipilih
List<Mahasiswa> dataFilter = widget.data
    .where((m) => m.jurusan == _selectedJurusan)
    .toList();

int totalFilter = dataFilter.length; //data jumlah mahasiswa
int lakiLaki = dataFilter
    .where((m) => m.gender
    .length; //data jumlah mahasiswa laki-laki
int perempuan = totalFilter - lakiLaki; //data jumlah mahasiswa perempuan
double rataIpkFilter =
    totalFilter ==
        0 //data rata-rata IPK
    ? 0
    : dataFilter.map((m) => m.ipk).reduce((a, b) => a + b) / totalFilter;

return Scaffold(
  appBar: AppBar(
    title: const Text(
      "Statistik Mahasiswa",
      style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),
    ),
    backgroundColor: Colors.green,
  ),
  body: SingleChildScrollView(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        //bagian ringkasan data global
        const Text(
          "Ringkasan Global",
          style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
        ),
        Row(
          children: [
            Expanded(
              child: _buildStatCardSmall(
                "Total Mahasiswa",
                "$totalGlobal",
                Icons.group,
                Colors.blue,
              ),
            ),
            Expanded(

```

```

        child: _buildStatCardSmall(
          "Rata-Rata IPK",
          rataIpkGlobal.toStringAsFixed(2),
          Icons.school,
          Colors.orange,
        ),
      ),
    ],
  ),
  const Divider(height: 40),

  //bagian data sesuai filter jurusan
  const Text(
    "Filter Program Studi",
    style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
  ),
  const SizedBox(height: 10),
  DropdownButtonFormField<String>(
    value: _selectedJurusan,
    decoration: const InputDecoration(
      border: OutlineInputBorder(),
      prefixIcon: Icon(Icons.category),
    ),
    items: _daftarJurusan
      .map((j) => DropdownMenuItem(value: j, child: Text(j)))
      .toList(),
    onChanged: (val) => setState(() => _selectedJurusan = val!),
  ),

  const SizedBox(height: 20),
  _buildStatCard(
    "Jumlah Mahasiswa",
    "$totalFilter",
    Icons.people,
    Colors.blue,
  ),
  _buildStatCard(
    "Laki-Laki",
    "$lakiLaki",
    Icons.male,
    Colors.lightBlue,
  ),
  _buildStatCard(
    "Perempuan",
    "$perempuan",
    Icons.female,
    Colors.pink,
  ),
  _buildStatCard(
    "Rata-Rata IPK",

```

```

        ratalPkFilter.toStringAsFixed(2),
        Icons.school,
        Colors.orange,
    ),
],
),
),
);
}

//widget untuk data sesuai filter jurusan
Widget _buildStatCard(
    String title,
    String value,
    IconData icon,
    Color color,
) {
    return Card(
        margin: const EdgeInsets.symmetric(vertical: 6),
        child: ListTile(
            leading: Icon(icon, color: color, size: 40),
            title: Text(title, style: const TextStyle(fontWeight: FontWeight.bold)),
            trailing: Text(
                value,
                style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
            ),
        ),
    );
}

//widget kecil untuk data global
Widget _buildStatCardSmall(
    String title,
    String value,
    IconData icon,
    Color color,
) {
    return Card(
        child: Padding(
            padding: const EdgeInsets.all(12.0),
            child: Column(
                children: [
                    Icon(icon, color: color),
                    Text(title, style: const TextStyle(fontSize: 12)),
                    Text(
                        value,
                        style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                    ),
                ],
            ),
        ),
    );
}

```

```

    ),
  );
}
}

```

Penjelasan :

Kode diatas adalah implementasi halaman antarmuka dinamis (StatefulWidget) bernama StatistikPage yang berfungsi sebagai pusat analitik dan visualisasi data aplikasi, di mana halaman ini menggabungkan interaksi pengguna melalui komponen dropdown untuk penyaringan (filtering), dengan kemampuan pengolahan Model Class untuk mengekstrak, mengkalkulasi, dan menyajikan ringkasan informasi metrik secara real-time baik dalam lingkup global (seluruh kampus) maupun spesifik berdasarkan program studi.

Berikut adalah rincian peran dan cara kerja dari masing-masing bagian pada kode statistik_page.dart :

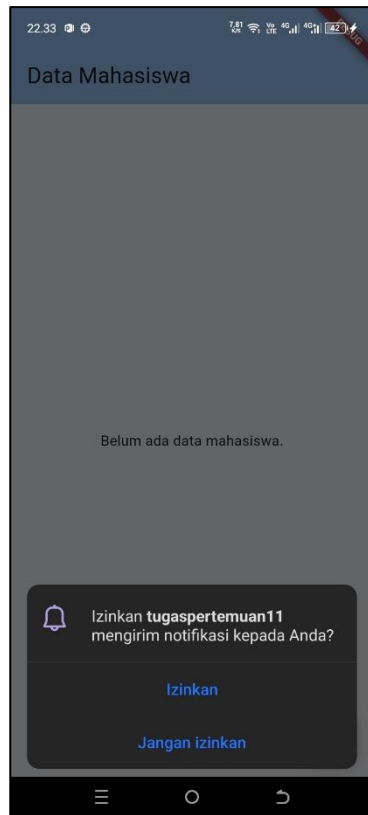
- **Penerimaan Data Terpusat (Sinkronisasi Data):** Melalui konstruktor required this.data, halaman ini menerima referensi list Mahasiswa yang dikelola di main.dart. Hal ini menjamin bahwa setiap penambahan atau perubahan data yang terjadi di halaman CRUD akan langsung tercermin pada perhitungan statistik di halaman ini tanpa perlu melakukan pengambilan data ulang.
- **Pengolahan Model Data Tingkat Lanjut:** Kode ini mendemonstrasikan bagaimana sebuah Model Class dimanipulasi untuk menghasilkan informasi baru. Menggunakan High-Order Functions dari Dart seperti .where() untuk memfilter data berdasarkan jurusan atau gender, serta kombinasi .map().reduce() untuk menjumlahkan seluruh IPK sebelum dibagi guna mendapatkan nilai rata-rata.
- **Interaksi dan Perubahan State:** Komponen DropdownButtonFormField digunakan sebagai alat interaksi utama pengguna. Ketika pengguna memilih program studi yang berbeda dari dropdown, properti onChanged akan memicu fungsi setState(). Fungsi ini memerintahkan Flutter untuk menghitung ulang seluruh variabel lokal dan merender ulang tampilan grafik agar sesuai dengan jurusan yang baru dipilih.
- **Logika Pengaman Komputasi (Safety Check):** Pada perhitungan rata-rata IPK (baik global maupun filter), terdapat logika Ternary Operator (totalGlobal == 0 ? 0 : ...). Ini adalah praktik pemrograman yang sangat penting untuk mencegah terjadinya error fatal berupa Division by Zero (pembagian dengan angka nol) apabila list mahasiswa masih kosong atau belum ada mahasiswa pada jurusan tertentu.
- **Modularitas Antarmuka (Reusable Widgets):** Untuk menjaga kode tetap rapi dan tidak berulang (DRY - Don't Repeat Yourself), elemen visual dipisahkan menjadi dua custom widget khusus, yaitu _buildStatCardSmall untuk metrik global di bagian atas (menggunakan Row dan Expanded), dan _buildStatCard untuk metrik spesifik di bagian bawah. Kedua widget ini menerima parameter teks, angka, ikon, dan warna agar tampilannya bisa disesuaikan secara dinamis.

8. Jalankan aplikasi menggunakan flutter run

b. Output

Berikut merupakan screenshot output :

1. Berikan izin notifikasi



2. Tampilan Halaman Menu Utama (Mahasiswa)



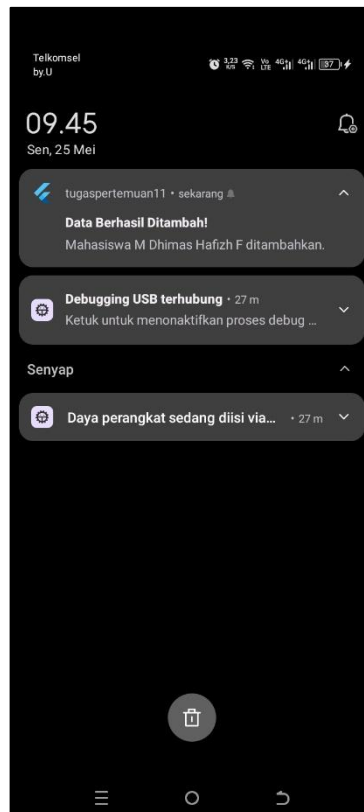
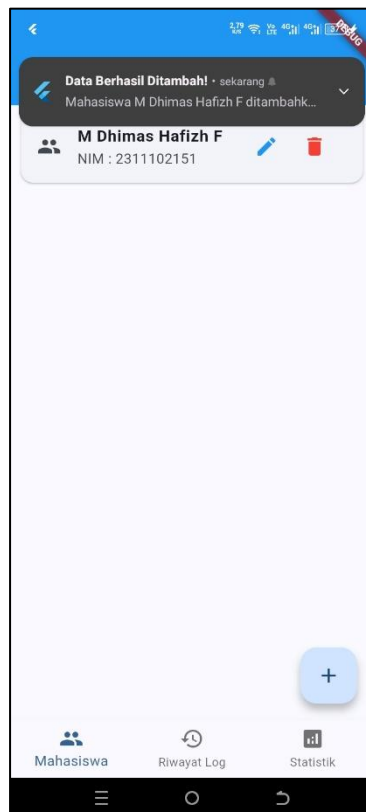
5. Tampilan form Tambah Data Mahasiswa

The screenshot shows a mobile application interface with a blue header bar labeled "Data Mahasiswa". Below the header is a white form titled "Tambah Mahasiswa". The form contains the following fields and options:

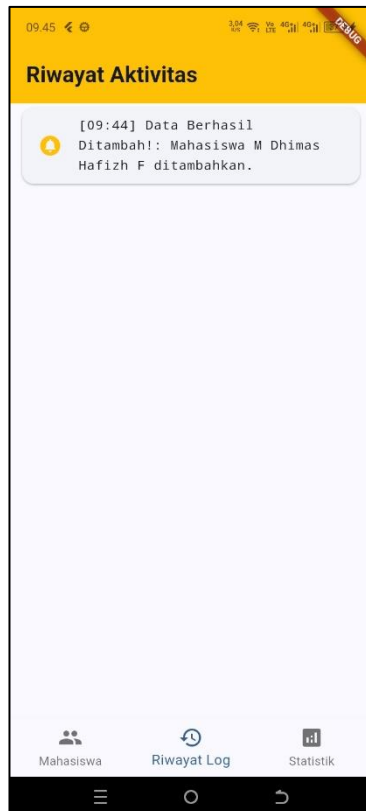
- Nama Lengkap: M Dhimas Hafizh F
- NIM: 2311102151
- Program Studi: Teknik Informatika (dropdown menu)
- IPK: 3.9
- Tanggal Lahir: 25/5/2004 (with a "Pilih" button)
- Jenis Kelamin: Laki-laki (selected with a radio button) and Perempuan (unselected)
- Buttons: "Batal" and "Simpan"

At the bottom of the screen is a navigation bar with three icons: "Mahasiswa" (selected), "Riwayat Log", and "Statistik".

6. Notifikasi ketika selesai menambahkan data



7. Tampilan Menu Riwayat Log Notifikasi (Notifikasi ketika menambah data mahasiswa tadi masuk disini)



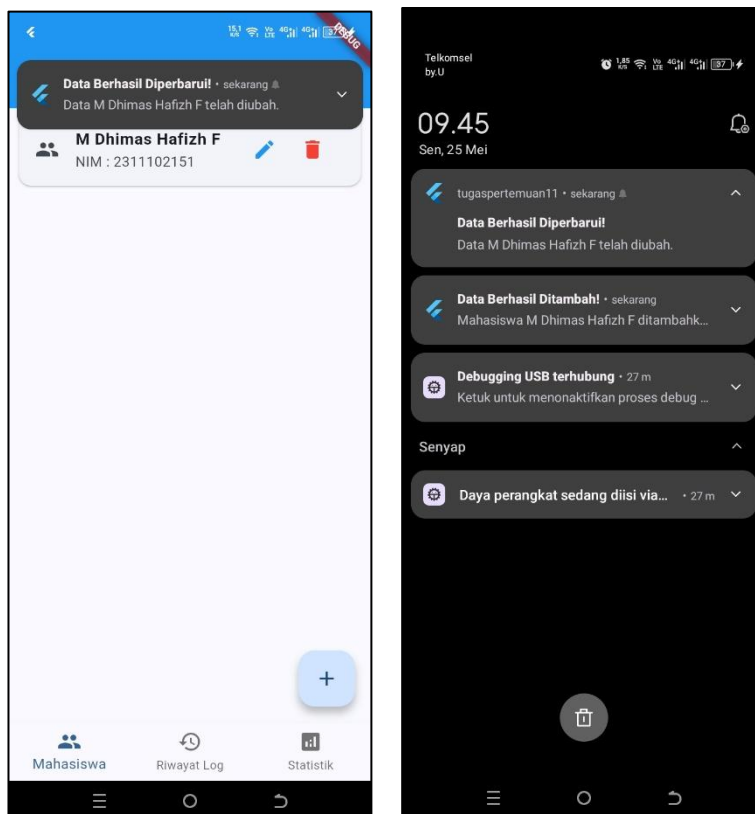
8. Tampilan detail data mahasiswa ketika card mahasiswa di halaman utama di tekan



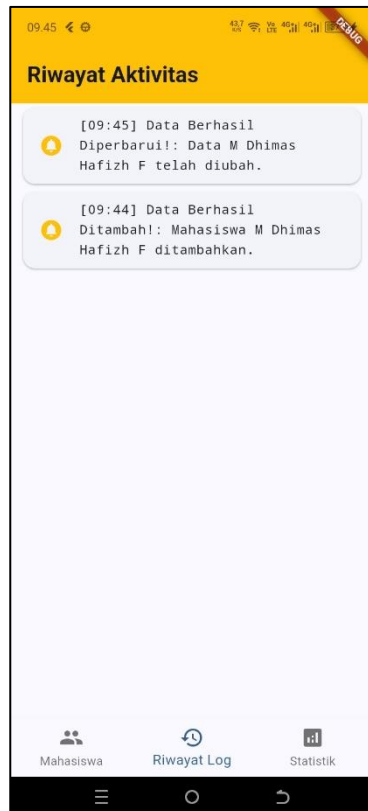
9. Tampilan form edit data mahasiswa

The screenshot shows a mobile application interface with a blue header titled "Data Mahasiswa". Below the header, there is a card for a student named "M Dhimas Hafizh F". A modal form titled "Edit Mahasiswa" is displayed in the center. The form contains the following fields: "Nama Lengkap" (M Dhimas Hafizh F), "NIM" (2311102151), "Program Studi" (Sistem Informasi), and "IPK" (3.7). Below these fields, there is a "Tanggal Lahir" field (25/5/2004) with a "Pilih" button, and a "Jenis Kelamin" section with radio buttons for "Laki-laki" (selected) and "Perempuan". At the bottom of the modal are "Batal" and "Simpan" buttons. The background shows a list of students and a bottom navigation bar with icons for "Mahasiswa", "Riwayat Log", and "Statistik".

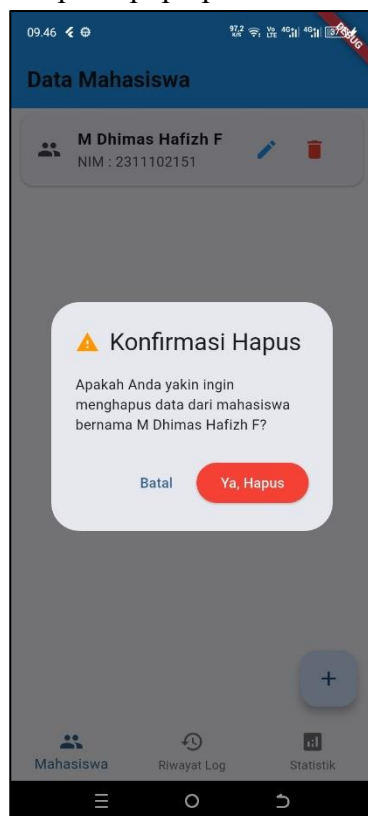
10. Notifikasi ketika selesai edit data mahasiswa



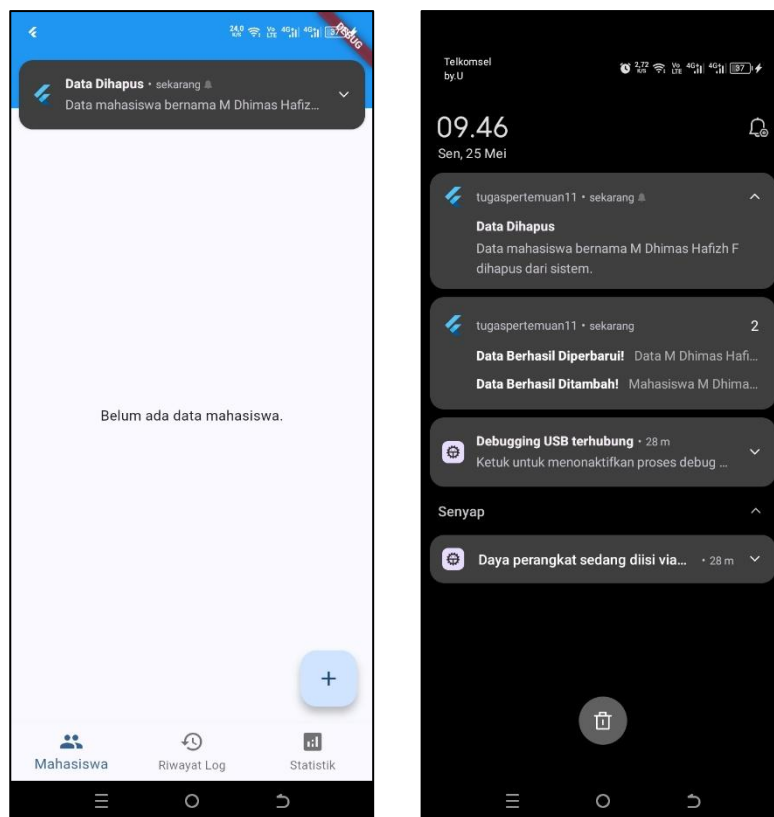
11. Tampilan Menu Riwayat Log Notifikasi (Notifikasi ketika edit data mahasiswa tadi masuk disini)



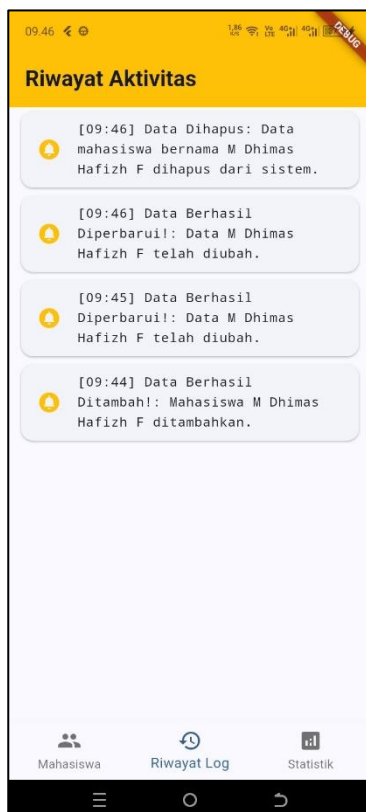
12. Tampilan pop up konfirmasi sebelum menghapus data



13. Notifikasi ketika selesai menghapus data



14. Tampilan Menu Riwayat Log Notifikasi (Notifikasi ketika hapus data mahasiswa tadi masuk disini)



15. Tampilan halaman menu statistik setelah ditambahkan data mahasiswa

